

DATA STRUCTURES — ZERO-SHOT, ONE-SHOT & FEW-SHOT PROMPTS

T. G. Sravan Teja | 261FB04042 | 1st MTech - CSE | Vignan University

1. Objective

Use AI to learn and revise important Data Structures topics through three prompting methods. Prompts can request definitions, examples, algorithms, complexity, applications and exam questions.

2. Prompt Types

Zero-Shot — Ask directly without giving an example. Best for concept explanations.

One-Shot — Give one example showing the required answer pattern. Best for consistent formatting.

Few-Shot — Give multiple examples showing the pattern, then ask for a new topic. Best for structured/pattern-based responses.

3. Important Data Structures Topics

Arrays • Linked Lists • Stacks • Queues • Recursion • Trees • Binary Search Trees • Heaps/Priority Queues • Graphs • Searching • Sorting • Hashing • Time & Space Complexity • Algorithms & Problem Solving

4. Zero-Shot Prompts

General: Explain [TOPIC] in Data Structures for a beginner-level MTech CSE student. Cover definition, key concepts, operations, a simple worked example, time and space complexity, applications, common mistakes and 3 exam questions. Use clear headings and concise language.

Algorithm: Explain the algorithm for [PROBLEM] step by step. Show pseudocode, a worked example, time complexity, space complexity and important edge cases.

Comparison: Compare [TOPIC A] and [TOPIC B] using definition, principle, operations, complexity, advantages, disadvantages and applications in a table.

Exam: Write a 10-mark answer on [TOPIC] with definition, representation/diagram in text, operations, algorithm, example, complexity, applications and conclusion.

5. One-Shot Prompts

Example format — Stack: Definition: LIFO linear structure. Operations: Push, Pop, Peek. Example: Push 10, 20, 30 → Pop removes 30. Complexity: Push/Pop typically $O(1)$. Application: Function-call management.

Prompt: Now explain [TOPIC] using exactly this structure: Definition → Key Concepts → Operations → Worked Example → Complexity → Applications → Common Mistakes → Exam Questions. Keep it technically accurate and student-friendly.

Code example format: Topic: Binary Search. Language: Python. Include a short implementation, explain the important lines, show sample input/output, and give time and space complexity. Now apply the same format to [TOPIC].

6. Few-Shot Prompts

Example 1 — Linear Search: Check elements from left to right until the target is found. Complexity: Best $O(1)$, Worst $O(n)$.

Example 2 — Binary Search: Repeatedly divide a sorted search range into two halves. Complexity: $O(\log n)$.

Example 3 — Bubble Sort: Compare adjacent elements and swap when out of order. Worst-case complexity: $O(n^2)$.

Prompt: Learn the response pattern from these examples and apply it to [TOPIC]. Include definition, core idea, operations/steps, worked example, pseudocode when useful, time complexity, space complexity, applications, limitations and 3 exam questions. Do not copy the examples; use them only as the answer pattern.

7. Topic-Specific Quick Prompts

Arrays: Explain traversal, insertion, deletion, searching and complexity.

Linked Lists: Explain singly, doubly and circular lists with insertion/deletion examples.

Stacks & Queues: Explain LIFO/FIFO, operations, implementations and applications.

Trees & BST: Explain terminology, traversals, insertion, deletion, searching, height and complexity.

Graphs: Explain representations, BFS, DFS, directed/undirected graphs and applications.

Sorting: Compare Bubble, Selection, Insertion, Merge, Quick and Heap Sort with complexities.

Hashing: Explain hash functions, collisions, chaining and open addressing.

Complexity: Explain Big-O, Big-Theta and Big-Omega with simple examples and comparisons.

8. Exam & Practice Master Prompts

Short Answer: Explain [TOPIC] in 5–7 exam-ready points. Include definition, key property, example, application and complexity.

Long Answer: Write a 10/15-mark answer on [TOPIC]. Include definition, diagram/representation, operations, algorithm/pseudocode, example, complexity, applications, advantages, limitations and conclusion.

Revision Sheet: Create a one-page revision sheet for [TOPIC] containing definitions, properties, operations, important complexities, algorithms, applications, common mistakes and memory tips.

Interview: Generate 10 interview questions on [TOPIC], from beginner to advanced, with concise answers and the concept tested.

Problem Practice: Generate 5 problems on [TOPIC] with increasing difficulty. Give the problem, expected concept, constraints and answer separately so the student can attempt first.

Code: Implement [OPERATION] using [LANGUAGE]. Explain the important code, sample input/output, edge cases, time complexity and space complexity.

9. Few-Shot Classification Prompt

Examples: Stack → Linear/LIFO; Queue → Linear/FIFO; Tree → Hierarchical; Graph → Network relationship. Task: Classify [NEW DATA STRUCTURE] into the most appropriate category and explain the reasoning, key property and typical use case. Do not force a category if the information is insufficient.

10. Data Structures Study Prompt

I am an MTech CSE student preparing Data Structures. Teach me [TOPIC] from basics to exam level. Start with the intuition, then definition, representation, operations, algorithm/pseudocode, worked example, complexity, applications, common mistakes and 5 practice questions. Use simple language but retain correct technical terminology.

11. Prompting Best Practices

- State the student level and purpose: learning, assignment, exam or interview.
- Specify the programming language whenever code is required.
- Ask for both time and space complexity for algorithms.
- Request a worked example rather than only a definition.
- For exams, specify 2-, 5-, 10- or 15-mark answer length.
- For diagrams, request ASCII/text diagrams when a visual tool is unavailable.
- Use examples in One-Shot/Few-Shot prompts to demonstrate the desired response pattern.
- Tell the AI not to invent algorithm properties or complexity values; state assumptions when necessary.

12. Reusable Prompt Template

Explain [TOPIC] for [LEVEL] students using [ZERO-SHOT / ONE-SHOT / FEW-SHOT] prompting. Include definition, key concepts, operations, example, algorithm/pseudocode, time complexity, space complexity, applications, limitations, common mistakes and exam questions. Use clear headings, concise language and technically accurate terminology.

13. Expected Learning Outcome

The student should be able to understand Data Structures concepts, compare alternatives, explain algorithms, analyze time/space complexity, write or understand implementations, solve problems and prepare structured exam answers.